

PisoFire on Kubernetes

SRE Project Report: Deployment, Resilience, Automation, and Observability

Universidade de Aveiro

João Varela (113780), Diogo Fernandes (114137), António
Alberto (114622)



Contents

1	Introduction	1
2	Product Scope and Core User Journey	2
2.1	Product Overview	2
2.2	Critical User Journey	2
3	Architecture and Stack Operation	3
3.1	Architecture	3
3.2	Kubernetes Resource Mapping	3
3.3	Main Data Flow	4
3.4	Operational Justification for the Technology Stack	6
4	Automation & Deployment Pipeline	7
4.1	Pipeline Overview	7
4.2	Configuration and Secret Handling	7
4.3	Bootstrap Jobs	8
4.4	Rollback Behaviour	8
5	Resilience, Scaling & Recovery	9
5.1	Stateless Application Redundancy	9
5.2	PostgreSQL High Availability with Patroni	9
5.3	Redis High Availability with Sentinel	10
5.4	Autoscaling	10
5.4.1	Horizontal Pod Autoscaler	10
5.4.2	HPA Demonstration	10
5.5	Disaster Recovery and Backup	11
5.5.1	Backup	11
5.5.2	Restore	11
6	Observability & Monitoring	12
6.1	Monitoring Stack	12
6.2	Application Metrics	12
6.2.1	Medusa Backend (/metrics)	12
6.2.2	Storefront (/api/metrics)	12
6.2.3	Exporters	13
6.2.4	Blackbox Synthetic Probes	13
6.3	Service Level Indicators and Objectives	13
6.3.1	Checkout Availability	13
6.3.2	Checkout Latency	13
6.3.3	Backend Error Rate	13
6.3.4	Storefront-to-Backend Health	13
6.3.5	Storefront-to-Backend Latency	13
6.3.6	Database Availability	14
6.3.7	Redis Availability	14
6.4	Alerting Rules	14
6.5	Grafana Dashboards	14
6.5.1	Long-Term Trend Monitoring	15

7	System Evaluation and Functional Verification	17
7.1	Health Checks	17
7.2	Smoke Test Procedure	17
7.3	Latest Validation Results	18
8	Architectural Review & Assessment	20
8.1	M1 Assumptions Revisited	20
8.1.1	Assumption: Single-instance PostgreSQL is sufficient	20
8.1.2	Assumption: Single-instance Redis is sufficient	20
8.1.3	Assumption: Fixed replica counts are adequate	20
8.1.4	Assumption: Basic health checks are sufficient	20
8.2	Known Limits of the Current HA Setup	21
9	Conclusion	22

Chapter 1

Introduction

PisoFire is a cloud-native e-commerce product for limited, high-demand retail sneaker releases and flash-sale traffic patterns: many users arrive at the same time, browse a small catalogue, add a size to the cart, and try to complete checkout before inventory disappears.

The project uses MedusaJS as the headless commerce backend and the Medusa Next.js starter as the storefront.

The final delivery contains a local Kubernetes baseline for `k3d` and a tenant overlay for the DETI cluster. The tenant overlay deploys the storefront, Medusa API, Medusa worker, Patroni-managed PostgreSQL, Redis Sentinel, MinIO, Prometheus Operator resources, Grafana dashboards, alert rules, backup jobs, and Horizontal Pod Autoscalers.

All scripts and manifests described here are present in the repository under `scripts/` and `k8s/tenant/`.

Chapter 2

Product Scope and Core User Journey

2.1 Product Overview

PisoFire is presented as a store for high-demand product releases. The intended business scenario is a flash-sale retail event where many customers enter the site during a short window and compete for limited inventory. The critical path is therefore the ability to keep product discovery, cart creation, checkout, payment session setup, and order creation available while the backing services are under pressure.

2.2 Critical User Journey

The primary critical user journey is checkout during a release:

1. A user reaches the storefront.
2. The storefront lists products.
3. The user selects a product variant and creates a cart.
4. The user adds a line item, enters address details, selects shipping, creates a payment collection, creates a payment session, and completes the cart.
5. Medusa persists the resulting order in PostgreSQL.

Chapter 3

Architecture and Stack Operation

3.1 Architecture

The system is split into stateless application tiers and stateful backing services. In the DETI tenant cluster, external traffic enters through Traefik Ingress hostnames and is routed to Kubernetes Services. The storefront talks to the Medusa backend through the internal `http://medusa:9000` Service URL. Medusa stores durable commerce data in PostgreSQL, uses Redis for caching, event bus, workflow engine, and distributed locking, and uses MinIO through the Medusa S3 file module.

The local baseline under `k8s/` targets a `k3d` cluster and is useful for development and repeatable smoke tests. The tenant overlay under `k8s/tenant/` targets the pre-created namespace `tenant-pisofire` in the DETI cluster and adds department-cluster-specific image names, Ingresses, monitoring resources, Redis Sentinel, and Patroni-managed PostgreSQL.

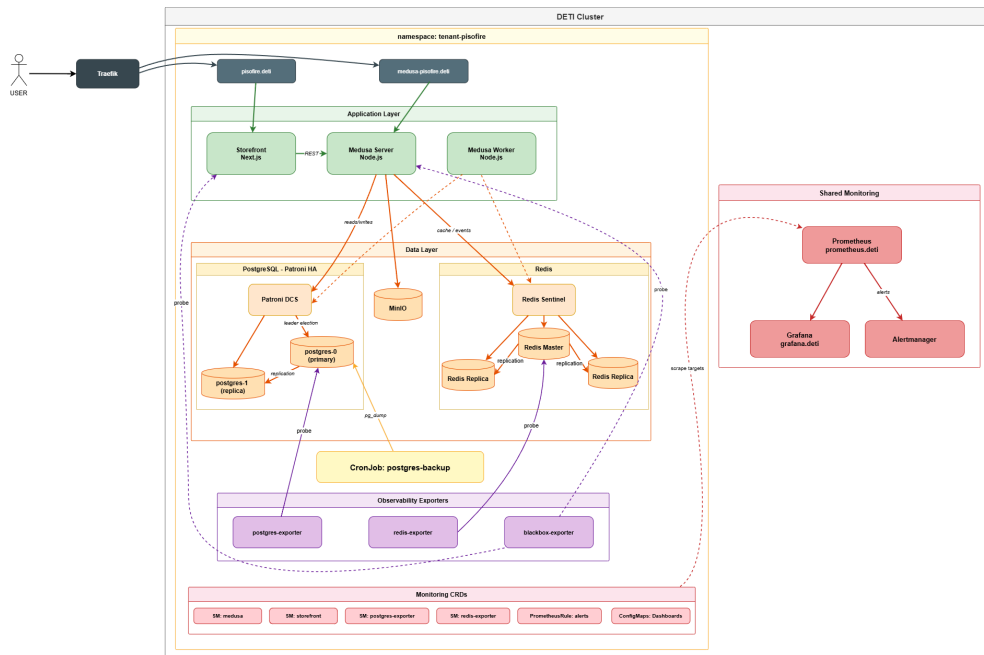


Figure 3.1: PisoFire architecture.

3.2 Kubernetes Resource Mapping

Component	Tenant Kubernetes Resources
-----------	-----------------------------

Configuration	ConfigMap/pisofire-config Secret/pisofire-secrets
Storefront	Deployment/storefront Service/storefront Ingress/pisofire-storefront
Medusa API	Deployment/medusa Service/medusa Ingress/pisofire-medusa
Medusa worker	Deployment/medusa-worker
PostgreSQL	StatefulSet/postgres Service/postgres Service/postgres-replica Service/postgres-headless ServiceAccount Role RoleBinding ConfigMap/postgres-patroni-config PVCs
PostgreSQL back-ups	CronJob/postgres-backup PVC/postgres-backups
Redis	StatefulSet/redis Service/redis Service/redis-sentinel Service/redis-headless PVCs
MinIO	Deployment/minio Service/minio Ingress/pisofire-minio PVC/minio-data Job/minio-setup
Bootstrap jobs	Job/bootstrap-store Job/bootstrap-admin
Autoscaling	HorizontalPodAutoscaler/medusa HorizontalPodAutoscaler/storefront HorizontalPodAutoscaler/medusa-worker
Monitoring	ServiceMonitor PrometheusRule Probe Exporter Deployments Grafana dashboard ConfigMaps

Alternative deployment option considered and discarded. An alternative deployment approach considered was packaging the system as a Helm chart instead of keeping plain Kubernetes manifests with Kustomize overlays and shell-based deployment scripts. Helm would reduce repetition, centralize parameter handling, and make multi-environment packaging easier. We discarded that option for this milestone because the deployment was still evolving and we prioritized transparency and debuggability over templating convenience. Plain manifests made every resource explicit, easier to inspect with `kubectl`, and simpler to relate directly to the architecture mapping in this report. This was particularly useful while iterating on stateful resources such as PostgreSQL, Redis, and the monitoring stack, where understanding the exact rendered objects mattered more than introducing another abstraction layer.

3.3 Main Data Flow

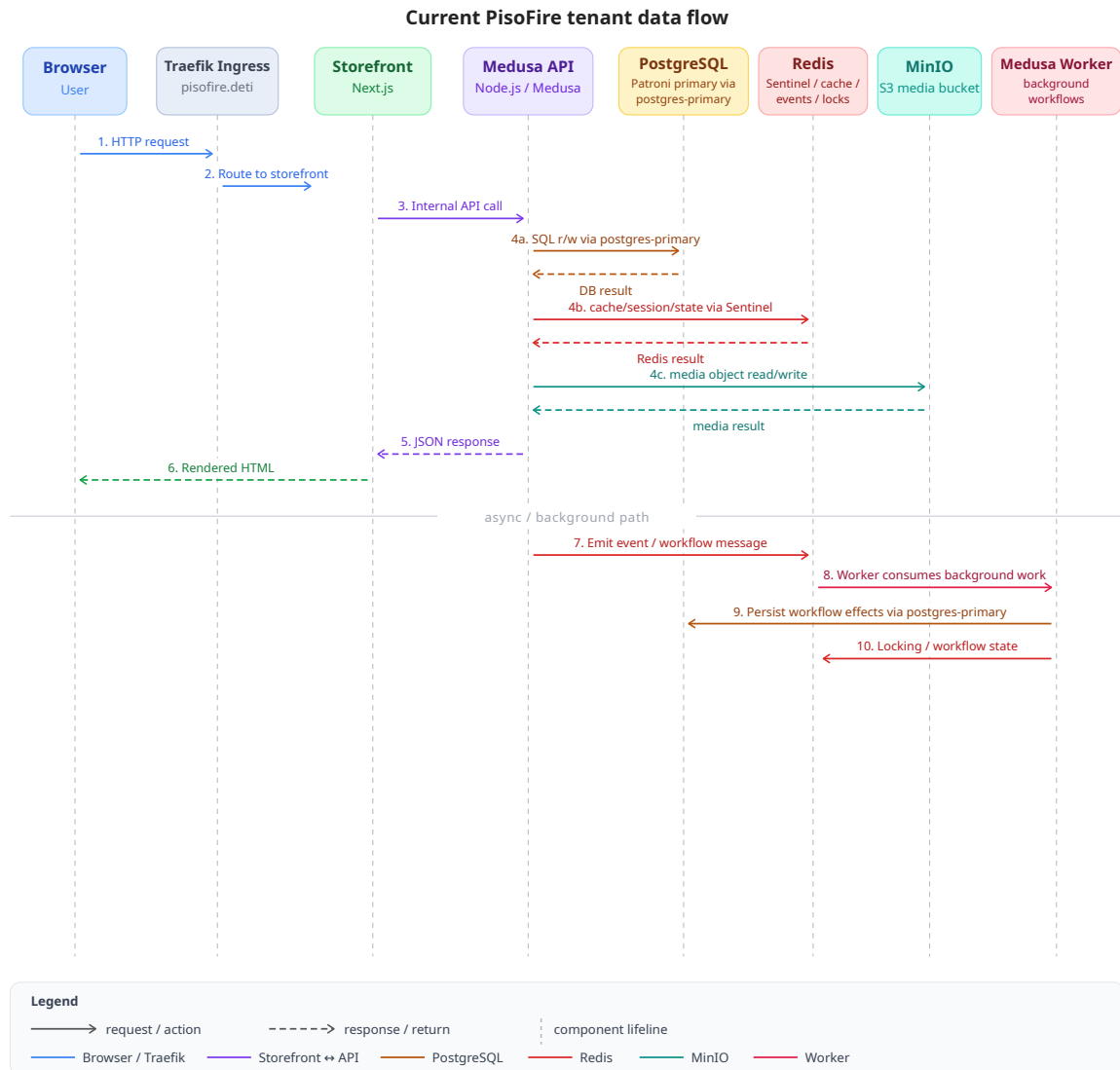
The storefront read path is:

1. The user opens <http://pisofire.deti>.

2. Traefik routes the request to the **storefront** Service.
3. The Next.js app calls Medusa through `MEDUSA_BACKEND_URL=http://medusa:9000`.
4. Medusa queries PostgreSQL for regions, products, variants, prices, carts, and orders.
5. Medusa uses Redis through Sentinel discovery for cache, events, workflow engine, and locking.
6. Product media can be served through MinIO when S3-backed file URLs are used.

The checkout write path is:

1. The storefront or smoke-test client obtains a publishable API key.
2. The client creates a cart and adds a variant as a line item.
3. Shipping, address, payment collection, and payment session calls are sent to Medusa.
4. Medusa writes the final order to PostgreSQL through the **postgres** Service, which follows the current Patroni primary.
5. Redis-backed framework modules handle cache, event, workflow, and locking concerns during the request lifecycle.



3.4 Operational Justification for the Technology Stack

The project selected each major component with an operational goal in mind, not only a development preference:

- **MedusaJS.** MedusaJS was chosen because it already provides the core commerce workflows needed for the project, including catalogue, cart, checkout, and order management, together with a compatible storefront starter. This let the team avoid spending most of the project on application development and instead focus on deployment, health checks, resilience, recovery, and observability.
- **Patroni for PostgreSQL high availability.** Patroni was chosen because it provides leader election, replica management, and failover behaviour suitable for PostgreSQL on Kubernetes without requiring an external distributed coordination system. It also exposes clear readiness and role information, which is useful for automation and operational validation.
- **Redis with Sentinel.** Redis is already a natural fit for Medusa’s cache, event, workflow, and locking needs. Sentinel was added because a single Redis pod would be an obvious operational weak point; Sentinel provides master discovery and failover while keeping the setup understandable and realistic for the course scope.
- **MinIO.** MinIO provides an S3-compatible object storage interface that fits well with Medusa’s file module and is straightforward to run inside Kubernetes.
- **Kustomize.** Kustomize was chosen to keep manifests plain and inspectable while still supporting environment-specific overlays, image tag changes, and Secret generation. This fits the project’s emphasis on understanding the deployed resources directly rather than hiding them behind a larger packaging layer.
- **Shell-script deployment automation.** A shell scripts gave us a simple, auditable way to automate build, deploy, validation, and rollback steps without introducing extra platform complexity.
- **Prometheus Operator and Grafana.** These tools are standard choices for Kubernetes observability and they are also given access to use by the Teacher, as they were available for the cluster.

Chapter 4

Automation & Deployment Pipeline

4.1 Pipeline Overview

M2 requires a single-script deployment, and the project implements exactly that: every step from building images to running Pods is driven by shell scripts checked into the repository. This keeps the deployment process inspectable and self-contained without requiring external services. The complete pipeline is:

1. **Edit code.**
2. **Build and push images:** `scripts/build-push-tenant-images.sh` builds the Medusa backend, Next.js storefront, and Patroni PostgreSQL images and pushes them to `registry.deti/tenant-pisofire` with the tag specified by the `IMAGE\_TAG` environment variable.
3. **Deploy:** `scripts/deploy-tenant-k8s.sh IMAGE\_TAG=<tag>` validates the kubeconfig context, checks the secrets file, rewrites image tags in the Kustomize overlay, applies the overlay with `kubectl apply -k`, waits for StatefulSet and Deployment rollouts, and restarts application Deployments to pick up the new image.
4. **Automatic rollback:** if any application Deployment rollout fails, the script calls `kubectl rollout undo` on all three Deployments (`medusa`, `medusa-worker`, `storefront`) and waits for the undo to complete. Recent events are printed for diagnosis.
5. **Validate:** `scripts/smoke-test-tenant-k8s.sh` runs the full end-to-end checkout journey and reports pass or fail.

Image versioning is controlled by the `IMAGE\_TAG` variable passed to both the build and deploy scripts. The deploy script rewrites the `newTag` fields in `k8s/tenant/kustomization.yaml` before applying, so every apply is pinned to an explicit image version and is fully reproducible.

4.2 Configuration and Secret Handling

Configuration is split between a ConfigMap and a Kubernetes Secret:

- **ConfigMap (pisofire-config):** non-sensitive runtime values, including `REDIS\_SENTINEL\_HOSTS`, `REDIS\_SENTINEL\_NAME`, CORS origins, public URLs, `S3\_ENDPOINT`, `S3\_BUCKET`, `POSTGRES\_USER`, `POSTGRES\_DB`, and `COOKIE\_SECURE`.
- **Secret (pisofire-secrets):** sensitive values generated by Kustomize from the git-ignored file `k8s/tenant/tenant-secrets.env`. The deploy script validates that all twelve required keys are present in this file before touching the cluster. If any key is missing the deploy aborts without applying any changes.

The twelve secret-backed keys are: `DATABASE\_URL`, `POSTGRES\_PASSWORD`, `POSTGRES\_REPLICATION\_PASSWORD`, `JWT\_SECRET`, `COOKIE\_SECRET`, `REVALIDATE\_SECRET`, `MEDUSA\_ADMIN\_EMAIL`, `MEDUSA\_ADMIN\_PASSWORD`, `MINIO\_ROOT\_USER`, `MINIO\_ROOT\_PASSWORD`, `S3\_ACCESS\_KEY\_ID`, and `S3\_SECRET\_ACCESS\_KEY`.

Because Kustomize regenerates the Secret on every deploy run, rotating a secret is as simple as updating `tenant-secrets.env` and rerunning `deploy-tenant-k8s.sh`. The new Secret value is applied atomically with the rest of the overlay and the Deployment rollout restart picks it up.

4.3 Bootstrap Jobs

Three one-shot Jobs (`bootstrap-store`, `bootstrap-admin`, `minio-setup`) run after each deploy to create or repair baseline store data and admin credentials. Kubernetes Jobs cannot be updated in place when image tags change, so the deploy script deletes and recreates them on every deploy.

4.4 Rollback Behaviour

Rollback is triggered automatically by the deploy script when a Deployment rollout does not complete within the timeout (default 300 seconds). The rollback procedure:

1. Calls `kubect1 rollout undo deployment/medusa deployment/medusa-worker deployment/storefront`.
2. Waits for the undo rollout to complete for each Deployment.
3. Prints recent namespace events for diagnosis.
4. Exits with a non-zero code so the operator knows the deploy failed.

StatefulSet rollouts (PostgreSQL, Redis) are not automatically rolled back because state migrations cannot be safely reversed without operator involvement. A failed StatefulSet rollout is reported and leaves the cluster in a partially-applied state, requiring manual recovery.

Chapter 5

Resilience, Scaling & Recovery

5.1 Stateless Application Redundancy

Both the Medusa API and the Next.js storefront start with two replicas and are configured for safe rolling updates:

- `maxUnavailable: 0` and `maxSurge: 1` - a new pod must be ready before any old pod is terminated, so the replica count never drops below the configured minimum during an update.
- Topology spread constraints on `kubernetes.io/hostname` with `whenUnsatisfiable: ScheduleAnyway` - the scheduler prefers to place replicas on different nodes; actual spreading depends on available cluster capacity.
- Dependency-aware readiness probes - Medusa pods are not marked ready until `/readyz` confirms PostgreSQL and Redis are reachable; storefront pods are not marked ready until `/api/ready` confirms the Medusa backend is reachable. This means traffic is only routed to pods that have verified their dependencies.

Pod anti-affinity is implemented through topology spread constraints rather than hard `podAntiAffinity` rules, keeping the setup portable to single-node test environments while still expressing the scheduling preference for distribution in multi-node clusters.

5.2 PostgreSQL High Availability with Patroni

PostgreSQL runs as a two-pod Patroni-managed StatefulSet. Patroni uses namespace-local Kubernetes endpoints and ConfigMaps as its distributed configuration store (DCS). It labels the writable pod `role=primary` and the standby `role=replica` via the Kubernetes API. Three Services expose the cluster:

- `postgres-primary`: selects `role=primary` - Medusa's `DATABASE_URL` points here.
- `postgres-replica`: selects `role=replica` - available for read-only workloads.
- `postgres-headless`: ClusterIP None, used by Patroni for internal pod-to-pod communication.

Patroni handles pod-level leader election automatically. When the primary pod is deleted, Kubernetes reschedules it, and Patroni elects a new primary (the former replica) while the restarted pod joins as a replica. The `postgres-primary` Service label selector always points to the current writer, so Medusa reconnects without any configuration change.

Failure injection. The resilience test deleted `postgres-1` (replica): no observable impact on storefront or Medusa health across 45 checks. Deleting `postgres-0` (primary): the storefront route and Medusa `/health` remained available; Medusa `/readyz` had five temporary 503 responses during the failover window before recovering to HTTP 200. Final state: `postgres-0` resumed as primary, `postgres-1` as replica, streaming replication lag 0 MB.

5.3 Redis High Availability with Sentinel

Redis runs as a three-pod StatefulSet. Each pod runs two containers: a Redis server and a Sentinel process. The Sentinel configuration uses quorum 2 and master name `pisofire-redis`. Medusa is configured with `REDIS\SENTINEL\HOSTS` and `REDIS\SENTINEL\NAME`, enabling `ioredis` master-role discovery and automatic client reconnection after a Sentinel-triggered failover.

The startup script (`redis-start.sh`) dynamically determines whether a pod should start as the master or a replica by querying existing Sentinels before writing `redis.conf`. Redis AOF persistence (`appendonly yes`, `appendfsync everysec`) is enabled so data survives pod restarts. Each pod has its own PVC on the Longhorn storage class.

If a replica pod fails, the remaining master/replica layout continues to serve traffic and the StatefulSet recreates the missing pod. If the master pod fails, Sentinel promotes one of the replicas to master and updates master discovery for clients. Because Medusa connects through Sentinel rather than a fixed Redis pod address, it can reconnect to the promoted master without manual reconfiguration, although a short failover window may still occur while leadership changes.

The smoke test validates: Sentinel returns a master address for `pisofire-redis`; exactly one Redis pod reports the `master` role; the other two report `slave/replica`.

5.4 Autoscaling

5.4.1 Horizontal Pod Autoscaler

Three HPAs are deployed targeting CPU and memory utilisation. No Vertical Pod Autoscaler is used: VPA would conflict with HPA on the same CPU/memory metrics and resource requests are stable enough not to require dynamic right-sizing.

Workload	Replica Range	Scale Conditions	Behaviour
medusa	2–5	CPU >70%, Memory >80%	Scale-up: stabilise 30s, max(+100% pods, +2 pods)/60s. Scale-down: stabilise 300s, max 50%/60s.
storefront	2–5	CPU >70%, Memory >80%	Same as Medusa. Fast up, conservative down suits flash-sale spikes.
medusa-worker	1–3	CPU >75%, Memory >80%	Scale-up: stabilise 60s, +1 pod/60s. Scale-down: stabilise 300s, -1 pod/120s.

The scale-up behaviour is intentional for the flash-sale scenario, as inventory windows are short and traffic arrives suddenly. The conservative scale-down (300-second stabilisation window) prevents thrashing when traffic tapers off.

5.4.2 HPA Demonstration

The HPA scale test was run against the tenant storefront using `scripts/hpa-scale-test-k8s.sh`. The controlled demo mode temporarily lowers HPA targets to 1% to guarantee scale-up regardless of cluster baseline load, generates in-cluster HTTP traffic with two BusyBox workers for 120 seconds, observes the HPA request the maximum replica count, then restores the original targets and replica count.

The actual terminal output from the tenant run is reproduced below:

```

Starting HPA scale demonstration
Namespace: tenant-pisofire
Target deployment: storefront
HPA: storefront
Initial replicas: 2
Temporarily lowering HPA targets to 1% for a deterministic demo.
Starting in-cluster HTTP load: workers=2 duration=120s
HPA storefront: desired=2 ready=2 cpu=6% memory=33%
HPA storefront: desired=2 ready=2 cpu=6% memory=33%
HPA storefront: desired=2 ready=2 cpu=64% memory=36%
HPA storefront: desired=4 ready=2 cpu=341% memory=41%
HPA scale test passed
Namespace: tenant-pisofire
Target deployment: storefront
HPA: storefront
Initial replicas: 2
Scale-up observed: yes
Pinning HPA to 2 replicas for cleanup
Restoring deployment/storefront replica count to 2
HPA cooldown: cpu=unknown%/70% memory=unknown%/80%
Restoring HPA targets for storefront

```

The controlled demo mode temporarily lowers HPA targets to 1

5.5 Disaster Recovery and Backup

5.5.1 Backup

A `postgres-backup` CronJob runs nightly at 02:00 UTC. It:

1. Waits for the `postgres-primary` Service to accept connections with `pg\_isready`.
2. Runs `pg\_dump -Fc -no-owner -no-acl` against the primary, writing to a timestamped file on the `postgres-backups` PVC.
3. Atomically renames the file (written first with a `.tmp` suffix to avoid partial reads).
4. Updates the `latest.dump` symlink.
5. Deletes dump files older than three days (`POSTGRES\_BACKUP\_RETENTION\_DAYS=3`).

The 26-hour alert threshold (`PisofirePostgresBackupStale`) means an alert fires if even one nightly run is missed, giving the team time to investigate before losing a second backup window.

Manual backups can be triggered at any time with `scripts/backup-postgres-k8s.sh`. The backup job writes to the `postgres-primary` Service, which Patroni keeps pointed at the current writer.

5.5.2 Restore

The restore script `scripts/restore-postgres-k8s.sh` requires an explicit confirmation value (`CONFIRM\_RESTORE=I\_UNDERSTAND\_THIS\_OVERWRITES\_POSTGRES`) and a kubeconfig context check before making any changes. The restore procedure:

1. Records current replica counts for `medusa`, `medusa-worker`, and `storefront`.
2. Scales all three application Deployments to zero and waits for pods to terminate.
3. Creates a one-shot restore Job that: waits for `postgres-primary` to be ready, drops and recreates the database, runs `pg\_restore`, and validates with `SELECT 1`.
4. Scales application Deployments back to their original replica counts and triggers a rollout restart.

Chapter 6

Observability & Monitoring

6.1 Monitoring Stack

The monitoring stack is built on the Prometheus Operator already deployed in the DETI cluster. The tenant overlay contributes:

- Four **ServiceMonitor** resources (Medusa, storefront, PostgreSQL exporter, Redis exporter) with a 15-second scrape interval.
- One blackbox exporter **Probe** resource covering four internal HTTP endpoints.
- One **PrometheusRule** resource with nine alert rules.
- Two Grafana dashboard **ConfigMaps**.
- Prometheus scrape annotations on all application Pods.

6.2 Application Metrics

6.2.1 Medusa Backend (/metrics)

All metrics use the prefix `pisofire\_medusa\_` and are registered on an isolated Prometheus Registry:

- `pisofire\_medusa\_http\_requests\_total\{method, route, status\_code\}` - request counter. Routes are normalised to two-segment paths (e.g. `/store/carts`) to avoid high-cardinality label explosion.
- `pisofire\_medusa\_http\_request\_duration\_seconds\{method, route, status\_code\}` - histogram with buckets from 5ms to 10s.
- `pisofire\_medusa\_dependency\_up\{dependency\}` - gauge for `database` and `redis`, set to 1 or 0 on every scrape.
- `pisofire\_medusa\_store\_entity\_count\{entity\}` - counts of publishable API keys, regions, products, and orders.
- `pisofire\_medusa\_info\{worker\_mode, node\_env\}` - process information gauge.
- Node.js default metrics (`pisofire\_medusa\_nodejs\*`) including heap usage, event loop lag, and GC statistics.

6.2.2 Storefront (/api/metrics)

- `pisofire\_storefront\_backend\_up` - 1 when `/api/ready` can reach Medusa `/readyz`, 0 otherwise.
- `pisofire\_storefront\_backend\_ready\_latency\_seconds` - histogram of the round-trip latency from the storefront to Medusa `/readyz`.
- `pisofire\_storefront\_info` - process information gauge.
- Node.js default metrics (`pisofire\_storefront\_nodejs\*`).

6.2.3 Exporters

- **postgres-exporter**: exposes `pg\_stat\_database\_\*`, connection counts, and replication lag metrics. Scraped by the `pisofire-postgres-exporter` ServiceMonitor.
- **redis-exporter**: exposes `redis\_memory\_used\_bytes`, `redis\_memory\_max\_bytes`, connected clients, and replication offset. Scraped by the `pisofire-redis-exporter` ServiceMonitor.

6.2.4 Blackbox Synthetic Probes

A blackbox exporter Deployment probes the following four internal endpoints every 15 seconds from within the cluster network, providing an independent external-perspective liveness check separate from the application's own metrics:

1. `http://storefront.tenant-pisofire.svc.cluster.local:8000/api/health`
2. `http://medusa.tenant-pisofire.svc.cluster.local:9000/health`
3. `http://medusa.tenant-pisofire.svc.cluster.local:9000/readyz`
4. `http://medusa.tenant-pisofire.svc.cluster.local:9000/store-readyz`

Metric relabelling maps each probe target URL to a short endpoint label (e.g. `endpoint="medusa-readyz"`) so Grafana panels can query `probe\_success\{endpoint="medusa-readyz"\}` rather than the full URL. This probing layer detects issues from a network perspective: a process that is up and self-reporting healthy but unreachable via Service DNS will still appear as a probe failure.

6.3 Service Level Indicators and Objectives

The following SLIs and SLOs focus on the checkout critical user journey. All PromQL expressions use the custom `pisofire\_\*` metrics.

6.3.1 Checkout Availability

SLI: Percentage of checkout-path backend requests that do not return a 5xx response.
SLO: At least 99.5% over the evaluation window.

6.3.2 Checkout Latency

SLI: p95 request duration for checkout-path backend routes.
SLO: Below 1 second.

6.3.3 Backend Error Rate

SLI: Percentage of all backend requests returning 5xx.
SLO: Below 5% over 5 minutes. (Also the threshold for the `PisofireHighErrorRate` alert.)

6.3.4 Storefront-to-Backend Health

SLI: `pisofire\_storefront\_backend\_up`.
SLO: Equal to 1 for at least 99.5% of the evaluation window.

6.3.5 Storefront-to-Backend Latency

SLI: p95 latency from the storefront metrics collection to Medusa `/readyz`.
SLO: Below 500 ms.

6.3.6 Database Availability

SLI: `pisofire\_medusa\_dependency\_up\{dependency="database"\}`.

SLO: Equal to 1 for at least 99.5% of the evaluation window.

6.3.7 Redis Availability

SLI: `pisofire\_medusa\_dependency\_up\{dependency="redis"\}`.

SLO: Equal to 1 for at least 99.5% of the evaluation window.

6.4 Alerting Rules

Nine alert rules are defined in the `pisofire-alerts` PrometheusRule, covering all SLO failure modes:

Alert	Severity	Condition and Rationale
<code>PisofireHighErrorRate</code>	warning	Backend 5xx rate > 5% for 2 minutes. Directly maps to the Backend Error Rate SLO.
<code>PisofireHighLatency</code>	warning	Backend p95 latency > 1s for 2 minutes. Maps to the Checkout Latency SLO.
<code>PisofireMedusaDown</code>	critical	Prometheus cannot scrape <code>/metrics</code> from the Medusa ServiceMonitor target for 1 minute.
<code>PisofirePostgresDown</code>	critical	PostgreSQL exporter target is unreachable for 1 minute, indicating the database may be unavailable.
<code>PisofirePostgresHighConnections</code>	warning	More than 80 active connections for 5 minutes. Guards against connection pool exhaustion (Patroni default <code>max_connections</code> is 100).
<code>PisofirePostgresBackupFailed</code>	critical	A backup Job has <code>status.failed > 0</code> for 5 minutes. Fires when a nightly dump fails.
<code>PisofirePostgresBackupStale</code>	warning	No successful CronJob run in more than 26 hours. Fires before a second window is missed.
<code>PisofireRedisDown</code>	critical	Redis exporter target is unreachable for 1 minute.
<code>PisofireRedisHighMemory</code>	warning	Redis memory usage exceeds 85% of <code>maxmemory</code> for 5 minutes.

6.5 Grafana Dashboards

Two Grafana dashboards are deployed via ConfigMaps:

PisoFire – Infrastructure (`uid: pisofire-infra`) covers cluster-level health:

- Running pod count and pod restarts over the last hour.
- Ready replicas per Deployment (Medusa, storefront) with colour thresholds.
- Last backup timestamp and PVC utilisation.
- Pod restarts over time as a time-series.
- CPU and memory usage per pod as time-series.

PisoFire – Service Health (`uid: pisofire-service`) covers application-level health:

- Medusa HTTP request rate and error rate.
- p95 request latency for checkout-path routes.

- Dependency health gauges (pisofire_medusa_dependency_up).
- PostgreSQL connection count and database size over time.
- Redis memory usage over time.
- Medusa Node.js heap usage and event loop lag.
- Probe success rate from the blackbox exporter.

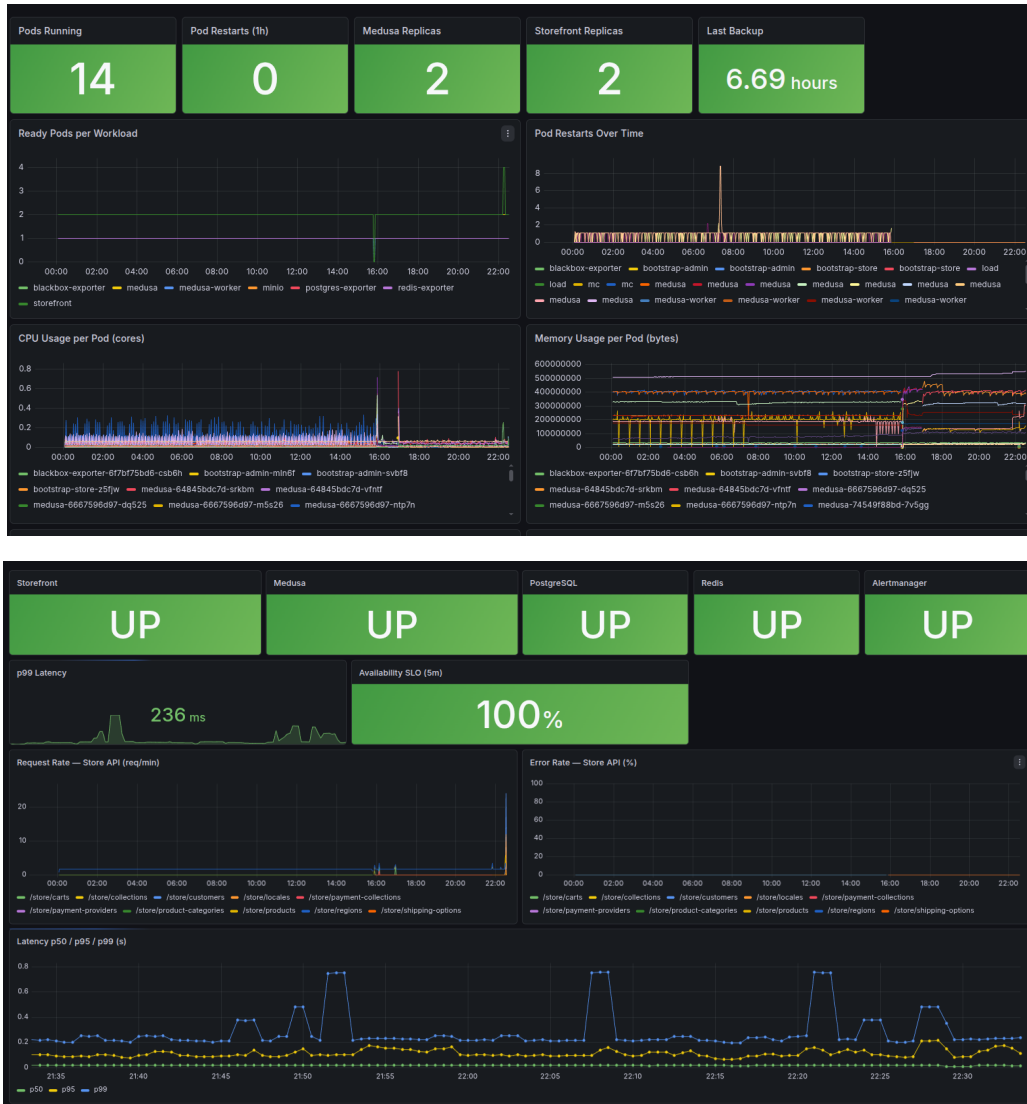


Figure 6.1: Grafana Dashboards

6.5.1 Long-Term Trend Monitoring

The dashboards are configured with a default time range of the last hour but can be zoomed out to days or weeks for trend analysis. The most operationally useful long-term panels are:

- **Pod restarts over time:** a rising restart count for a specific pod over days indicates a recurring OOM kill or crash loop that individual moment-in-time checks miss.
- **PVC utilisation:** the backup PVC will grow until the three-day retention window stabilises. Monitoring its size over time detects unexpected growth from failed retention logic.
- **PostgreSQL connection count:** a slow upward trend indicates a connection leak in Medusa or the worker (no PgBouncer is deployed).

- **Redis memory:** a growing baseline without a corresponding increase in product catalogue or cart volume suggests a cache TTL problem or event bus subscriber backlog.
- **Node.js heap usage:** a monotonically increasing heap in the Medusa process across restarts indicates a memory leak. The Medusa worker is particularly relevant here since it is long-running and handles background workflow execution.

Chapter 7

System Evaluation and Functional Verification

7.1 Health Checks

Component	Probe or Check	Purpose
Medusa API	startup/liveness /health; readiness /readyz	Separates process health from dependency-aware readiness.
Medusa worker	startup/liveness/readiness /health; init container waits for Medusa /readyz	Prevents the worker from starting before the backend dependencies are ready.
Storefront	startup/liveness /api/health; readiness /api/ready	Keeps traffic away from storefront pods that cannot reach a ready backend.
PostgreSQL	Patroni REST API /readiness and /liveness on port 8008	Lets Kubernetes check Patroni-managed PostgreSQL state instead of only checking TCP reachability.
Redis	redis-cli ping for both Redis and Sentinel containers	Verifies Redis and Sentinel processes respond locally.
MinIO	/minio/health/ready and /minio/health/live	Checks S3-compatible object storage readiness and liveness.

Readiness is now stronger than the original M1 baseline because it validates PostgreSQL and Redis dependencies. It still does not validate a specific application version, image tag, or database schema version.

7.2 Smoke Test Procedure

The tenant smoke test is the main repeatable validation artifact. It checks:

- safe kubeconfig context;
- StatefulSet and Deployment rollouts;
- Patroni primary and replica labels;
- PostgreSQL recovery state on primary and replica;
- bootstrap job completion;
- HPA resources and metrics API availability;

- backup CronJob and backup PVC;
- Redis Sentinel master discovery and role count;
- Medusa and storefront health through Ingress;
- Medusa and storefront metrics endpoints;
- Medusa Admin login and `/admin/users/me`;
- full Store API checkout journey ending in order creation.

```
Waiting for core workloads in namespace tenant-pisofire
Checking PostgreSQL Patroni primary and replica roles
NAME      READY   STATUS    RESTARTS   AGE   LABELS
postgres-0 1/1     Running   0           6h34m app=postgres,apps.kubernetes.io/pod-index=0,cluster-name=pisofire-postgres
,controller-revision-hash=postgres-845b78bdc6,role=primary,statefulset.kubernetes.io/pod-name=postgres-0
postgres-1 1/1     Running   0           6h36m app=postgres,apps.kubernetes.io/pod-index=1,cluster-name=pisofire-postgres
,controller-revision-hash=postgres-845b78bdc6,role=replica,statefulset.kubernetes.io/pod-name=postgres-1
Waiting for bootstrap job completion
Checking autoscaling and disruption controls
PodDisruptionBudgets are not present; tenant RBAC does not allow creating them.
Checking Redis Sentinel and replication roles
Checking backend and storefront health through Ingress
Checking Medusa admin authentication through Ingress
Running end-to-end store API journey
Tenant smoke test passed
Context: tenant-pisofire-context
Namespace: tenant-pisofire
Backend health: 200
Backend metrics: 200
Storefront status: 307
Storefront metrics: 200
Autoscaling controls: present
PostgreSQL backup controls: present
PostgreSQL Patroni primary: postgres-0
PostgreSQL Patroni replicas: 1
Stateful disruption controls: present
Redis Sentinel master: redis-2.redis-headless.tenant-pisofire.svc.cluster.local
Redis Sentinel HA: one master, two replicas
Admin authentication: passed
Order: order_01KTA96T5XZ8QK5B24ADNRBY66
```

7.3 Latest Validation Results

The latest tenant validation showed:

- Tenant smoke test passed.
- Backend health returned HTTP 200.
- Backend metrics returned HTTP 200.
- Storefront returned HTTP 307, which is acceptable because the storefront can redirect during locale or region setup.
- Storefront metrics returned HTTP 200.
- Admin authentication passed.
- Store API checkout journey passed and created an order.
- Patroni primary was `postgres-0`; one replica was available.
- Redis Sentinel reported one master and two replicas.

The HPA demonstration also passed for the tenant storefront. Desired replicas rose from 2 to 5 during the controlled load phase, and the script restored the original HPA configuration afterwards.

“latex

Table 7.2: Summary of Kubernetes resources status

Resource type	Total	Active/Available	Notes
Pods	19	14 Running	5 Completed
Services	13	13	All services are ClusterIP
Deployments	7	7 available	All replicas are ready
ReplicaSets	26	7 active	Several inactive ReplicaSets from previous rollouts
StatefulSets	2	2 available	PostgreSQL and Redis
HPAs	3	3 active	Medusa, Medusa Worker and Storefront
CronJobs	1	1 active	Daily PostgreSQL backup
Jobs	5	5 completed	Bootstrap, MinIO setup and backup jobs

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
blackbox-exporter-6f7bf75bd6-csb6h	1/1	Running	0	2d10h	10.42.5.219	k3s-worker-01	<none>	<none>
bootstrap-admin-mln6f	0/1	Completed	0	5h49m	10.42.15.107	k3s-worker-08	<none>	<none>
bootstrap-store-jkrz8	0/1	Completed	0	5h49m	10.42.12.15	k3s-worker-07	<none>	<none>
medusa-64845bdc7d-srkbm	1/1	Running	0	5h49m	10.42.15.108	k3s-worker-08	<none>	<none>
medusa-64845bdc7d-vfntf	1/1	Running	0	5h48m	10.42.12.17	k3s-worker-07	<none>	<none>
medusa-worker-66dd68dd8b-n2h7g	1/1	Running	0	5h49m	10.42.12.16	k3s-worker-07	<none>	<none>
minio-79d4f767d4-lnqdz	1/1	Running	0	2d10h	10.42.6.101	k3s-worker-03	<none>	<none>
minio-setup-9lvjf	0/1	Completed	0	5h49m	10.42.5.20	k3s-worker-01	<none>	<none>
postgres-0	1/1	Running	0	6h40m	10.42.18.122	k3s-worker-09	<none>	<none>
postgres-1	1/1	Running	0	6h41m	10.42.12.14	k3s-worker-07	<none>	<none>
postgres-backup-29674200-bbsqx	0/1	Completed	0	43h	<none>	k3s-worker-04	<none>	<none>
postgres-backup-29675640-hltqg	0/1	Completed	0	19h	10.42.12.8	k3s-worker-07	<none>	<none>
postgres-exporter-c67dd4bfc-s6qwm	1/1	Running	0	2d10h	10.42.8.14	k3s-worker-05	<none>	<none>
redis-0	2/2	Running	0	2d10h	10.42.6.105	k3s-worker-03	<none>	<none>
redis-1	2/2	Running	0	2d10h	10.42.7.25	k3s-worker-04	<none>	<none>
redis-2	2/2	Running	0	2d10h	10.42.18.76	k3s-worker-09	<none>	<none>
redis-exporter-5cccd4ff7f-xlzn5	1/1	Running	0	2d10h	10.42.12.234	k3s-worker-07	<none>	<none>
storefront-76fcfb489c-8rh74	1/1	Running	0	10m	10.42.15.148	k3s-worker-08	<none>	<none>
storefront-76fcfb489c-f6td2	1/1	Running	0	10m	10.42.10.90	k3s-worker-06	<none>	<none>

Chapter 8

Architectural Review & Assessment

8.1 M1 Assumptions Revisited

8.1.1 Assumption: Single-instance PostgreSQL is sufficient

M1 state: A single Kubernetes Deployment backed by one PVC. Any pod failure meant the database was unavailable.

Why it was wrong: A single-pod database has no redundancy. A node failure, eviction, or OOM kill takes the entire commerce backend offline and the checkout SLO fails immediately. The smoke test would pass on a healthy cluster but offered no evidence of availability under failure.

Change made: Replaced with a two-pod Patroni-managed StatefulSet. Patroni provides automatic leader election using Kubernetes endpoints and ConfigMaps as DCS, streaming replication, and automatic failover at the pod level. The `postgres-primary` Service always selects the current writer by label, so Medusa reconnects without configuration changes.

8.1.2 Assumption: Single-instance Redis is sufficient

M1 state: A single Redis pod. A pod failure meant the Medusa cache, event bus, workflow engine, and distributed locking were all unavailable, causing Medusa API errors.

Why it was wrong: Medusa uses Redis for multiple subsystems; any Redis outage produces cascading failures in the application tier even when PostgreSQL is healthy.

Change made: Replaced with a three-pod StatefulSet where each pod runs both a Redis server and a Sentinel process. Sentinel quorum 2 provides automatic master failover. The `ioredis` client in Medusa uses Sentinel master-role discovery, so the application reconnects to the promoted master.

8.1.3 Assumption: Fixed replica counts are adequate

M1 state: Two replicas for Medusa and storefront. Flash-sale traffic spikes would exhaust capacity with no automatic response.

Why it was wrong: PisoFire's business scenario is defined by sudden demand spikes at release time. Fixed replicas cannot handle this without significant over-provisioning at all other times.

Change made: HPAs with behaviour policies (aggressive scale-up, conservative scale-down) allow the system to respond to load within one polling cycle (30-second stabilisation window).

8.1.4 Assumption: Basic health checks are sufficient

M1 state: Liveness on `/health` only. Pods were considered ready as soon as the process started, even if PostgreSQL was still migrating or Redis was not yet reachable.

Why it was wrong: During startup, Medusa pods would receive traffic before dependencies were ready, producing 5xx responses for real users.

Change made: A separate `/readyz` endpoint checks PostgreSQL (via a live query) and Redis (via PING) before reporting ready. The storefront's `/api/ready` calls Medusa's `/readyz`, creating a dependency chain that gates traffic at both tiers. Medusa worker pods use an init container that blocks until Medusa `/readyz` returns 200.

8.2 Known Limits of the Current HA Setup

The current tenant deployment is highly available at the application and core data-service level, but it is not a complete production-grade HA platform. The main limits are:

- **PostgreSQL failover is not zero-loss.** Patroni provides automatic leader election and pod-level failover, but replication is asynchronous. If the primary fails before its latest WAL has reached the replica, the most recent committed transactions can be lost.
- **Disaster recovery is operator-driven.** Nightly backups and a restore script exist, but restore is a manual DR action. There is no automatic database rebuild-and-restore loop after storage corruption or catastrophic state loss.
- **Health checks are dependency-aware but not version-aware.** Readiness proves that PostgreSQL and Redis are reachable, but it does not validate that the running image tag, application version, or database schema version matches the intended deployment state.
- **Rollouts are controlled but somewhat basic.** Stateless services use RollingUpdate with automatic rollback on failed Deployment rollouts, which gives predictable updates for this project. More advanced strategies such as canary or progressive delivery are not included, and stateful updates still require closer operator supervision because data and schema changes must be handled carefully.
- **Autoscaling is still resource-based.** The HPAs react to CPU and memory utilisation, which is a reasonable baseline and works for the current deployment, but it is not the smartest signal for an e-commerce workload. A more mature setup would also consider application-level indicators such as request rate, queue depth, checkout latency, or error rate so scaling decisions match real user pressure more closely.

Chapter 9

Conclusion

PisoFire at Milestone 2 satisfies all four M2 operational areas. The deployment pipeline is fully automated, a single script builds images, validates secrets, applies the Kustomize overlay, waits for rollouts, and triggers automatic rollback on failure. The stateless tiers (Medusa API, worker, storefront) are replicated, autoscaled with behaviour policies suited to flash-sale traffic patterns, and traffic-gated by dependency-aware readiness probes. PostgreSQL runs under Patroni with automatic pod-level failover, and Redis runs with Sentinel-based master election. Nightly backups are automated and alerting fires if they fail or become stale. The observability stack covers application metrics, infrastructure metrics, synthetic HTTP probes, SLI-aligned alert rules, and two Grafana dashboards with long-term trend panels.